.SQL Queries Document

Setayesh Saeidi - 4013185

1. Products in a Category and All Subcategories Sorted by Price Descending

What it does:
Finds all products under a given category (e.g., "Electronics") including every level of its subcategories, and returns their titles and prices sorted with the highest price first.

Key techniques:

- `WITH RECURSIVE` builds the full category subtree. That's needed because categories are hierarchical; we want not only direct children but grandchildren, etc.
- Joining the recursive set (`cat_tree`) to `product_categories` and then `products` filters to all products in that whole subtree.
- `DISTINCT` avoids duplicates if a product is in multiple matching categories.

Real-world purpose:
Useful in a product catalog UI when showing "All Electronics" where you want to include stuff nested beneath the category (like chargers, cables) and highlight premium/high-priced items first. Also good for analytics like "what are the most expensive products in this entire branch."

Note: Recursive CTE is the natural way in SQL to walk a tree structure without writing application code to traverse it manually.

---

2. Average Rating per Product (only with at least one review)

What it does:
Computes, per product that has reviews, the average rating (rounded) and how many reviews it has. Sorted highest-rated first.

Key techniques:

- Inner join to `reviews` automatically excludes products with zero reviews.
- `AVG(r.rating)` aggregated per product and rounded to two decimals for presentation.
- `HAVING COUNT(*) >= 1` is redundant because the join already ensures at least one, but it documents the intention.

- Grouping by product title (could use id + title in more robust version).

Real-world purpose:
Used in recommendation dashboards or product detail pages to rank products by perceived quality, e.g., “Top-rated products” or filtering to show “products with reviews and their average rating.”

---

3. Users Who Applied Discounts, Count and Distinct Codes

What it does:
Lists users (by email) who used discounts on their orders, showing total discount usages and number of distinct discount codes they used. Sorted ascending by total usages.

Key techniques:

- Filters `orders` with `WHERE o.discount_id IS NOT NULL` to consider only orders where a discount was applied.
- `COUNT(o.discount_id)` gives total times they used any discount; `COUNT(DISTINCT o.discount_id)` shows code variety.
- Grouping by email yields one row per user.

Real-world purpose:
Useful for marketing analysis: identify users who rely on repeated discounts vs. those experimenting with different codes. Can inform loyalty program design or abuse detection (e.g., too many distinct discounts).

---

4. Products with Specific Attribute, Ordered ≥2 Times, + Stock

What it does:
Finds products with a given EAV attribute (example: Weight = '0.050') that appear in at least two distinct orders, returns each product’s title, number of distinct orders containing it, and current stock.

Key techniques / unique design:

- Joins through `product_attribute_values` and `product_attributes` to apply the EAV filter.
- Aggregates over `order_items` to count distinct orders (`HAVING COUNT(DISTINCT oi.order_id) >= 2`).

- Uses `COALESCE(inv.quantity,0)` by pulling inventory with a left join to safely handle missing inventory.
- Packs per-product info plus the threshold filtering into one CTE (`qualified`) and then selects from it, avoiding separate queries.

Real-world purpose:
Helps supply chain or product managers know which attribute-specific products (e.g., a particular weight class) have some traction (ordered multiple times) and what their current availability is—good for inventory replenishment decisions.

---

5. Products That a User Ordered and Reviewed (Matching User)

What it does:
Finds cases where a user both ordered a product and reviewed it. Returns product title, username, rating, and the order date.

Key techniques:

- Starts from `reviews` to anchor on the existence of a review.
- Joins `order_items` and `orders` with an extra condition `o.user_id = u.id` to ensure the reviewer is the same person who placed the order.
- Orders by the `placed_at` to see the timeline.

Real-world purpose:
Useful for verifying review authenticity (did the reviewer actually purchase the product), or for personalized follow-ups like “thank you for reviewing what you bought,” or identifying high-engagement customers.

---

6. Products with Stock Below Average Inventory

What it does:
Selects all products whose current inventory quantity is below the average inventory quantity across all products. Returns title, stock, and price sorted by lowest stock first.

Key techniques:

- CTE `avg_stock` computes the average once.
- Join condition `inv.quantity < a.avg_qty` filters below-average.
- Joins to `products` to get titles and prices.

Real-world purpose:
Inventory alerting: detect potential stockouts before they happen; useful for automatic reorder triggers or dashboards showing “low-performing stock levels” relative to the catalog average.

---

7. Trigger to Deduct Inventory When Order Item Inserted

What it does:
Defines a trigger function that, upon insertion of a new `order_items` row, decrements the corresponding `inventory.quantity` by the ordered amount with safety checks.

Key techniques:

- `FOR UPDATE` locks the inventory row to avoid race conditions in concurrent inserts.
- Validation: rejects non-positive quantities, missing inventory row, and insufficient stock with clear exceptions.
- Updates the `inventory` and sets `last_updated` timestamp.
- Trigger is `AFTER INSERT` so it reacts immediately after a new order item is created.
- Test section sets up a known scenario (test product, inventory, order) and validates the stock change.

Real-world purpose:
Automatic stock management when orders are placed to keep inventory accurate. This is critical in e-commerce to prevent overselling and maintain real-time availability.

---

8. Revenue and Units Sold per Category for Delivered Orders

What it does:
For each category, calculates total revenue (`quantity * price_each`) and total units sold from orders with status `'Delivered'`.

Key techniques:

- Joins through category→product→order_items→orders.
- Filters on `o.status = 'Delivered'` to only count completed sales.
- Aggregates revenue and quantity, uses `HAVING` to exclude categories with zero sold units.
- Rounds revenue for cleaner presentation.

Real-world purpose:
Category-level performance reporting: see which categories bring in most money and volume, useful for merchandising, inventory allocation, or promotional planning.

---

9. Users Who Spent More Than Average Spent

What it does:
Identifies users whose total spending (sum of `orders.total_amount`) exceeds the average total spent per user. Returns username, total spent, and their order count.

Key techniques:

- First CTE computes per-user totals.
- Second CTE calculates the overall average of those totals.
- Final filter selects users above that average.

Real-world purpose:
Customer segmentation: find high-value customers for retention campaigns, upsells, or VIP rewards.

---

10. Products with Attribute Combo, Ordered ≥3 Times, Qty >5, Reviewed, Category Count

What it does:
Finds products that have a specific combination of EAV attributes (e.g., `Color='Black'` and `Weight=1.0`), have appeared in at least three distinct orders with total quantity >5, have at least one review (so average rating exists), and reports their title, total ordered quantity, average rating, and number of categories.

Key techniques:

- Multiple filtered CTEs:
  - `filtered_products` builds the intersection of two attribute conditions.
  - `order_stats` enforces order count and quantity thresholds.
  - `review_stats` gives average rating (ensures at least one review).
  - `category_counts` captures how widely the product is categorized.
- Final join combines all conditions so only fully qualifying products remain.

Real-world purpose:
Complex product quality + demand filter: e.g., highlight well-reviewed, in-demand premium variants for upselling, or decide which multi-attribute SKUs to stock more heavily.

---

11. Premium Users' Cart Summary

What it does:
For premium users, computes how many items are in their cart and the total cart value (quantity × price), including only those with >0 items.

Key techniques:

- Filters premium users via `WHERE u.is_premium = true`.
- Joins carts → cart_items → products to compute item counts and monetary value.
- `HAVING SUM(ci.quantity) > 0` ensures non-empty carts.
- Rounds the value.

Real-world purpose:
Targeted cart recovery or upsell: identify premium users with high cart values for prompts ("complete your purchase") or to offer extra incentives.

---

12. Products with More Reviews Than Average

What it does:
Lists products whose review count exceeds the average number of reviews per product, showing title, review count, and stock.

Key techniques:

- `review_counts` gets per-product review totals, including zeros.
- `avg_reviews` computes average across all products.
- Filtering joins ensure only products with review_count > average and at least one review show up.
- Stock is joined in, using `COALESCE` semantics via presence in inventory.

Real-world purpose:
Spotlight popular products (by engagement, not just rating) that might deserve prioritization in marketing or need stock scaling based on visibility.

---

13. Trigger to Increment Discount Usage and Log It

What it does:
On insertion of an order with a non-null `discount_id`, increments that discount's `usage_count` and writes a detailed log entry (user, discount, order, timestamp).

Key techniques:

- Guard `WHEN (NEW.discount_id IS NOT NULL)` avoids unnecessary function calls.
- Updates the base `discounts.usage_count` atomically.
- Separately logs the usage in `discount_usage_log` for auditability.
- Uses `CURRENT_TIMESTAMP` for usage time.

Real-world purpose:
Keeps accurate metrics for how many times each discount is used (for campaign effectiveness) while retaining a per-use audit trail for debugging abuse or doing attribution.

---

14. Per-User Total Items Ordered and Total Spent (Fixing Duplication Bug)

What it does:
For each user with at least one order, it computes total quantity of items ordered across all their orders and total money spent, avoiding double counting.

Key techniques / fix rationale:

- Separates item aggregation (`user_item_counts`) from monetary aggregation (`user_spent`) to prevent multiplicative inflation that happens when joining orders and order_items directly and summing both in one go.
- Joins back to users to show email.
- `COALESCE` protects against users with orders but no items.

Real-world purpose:
Customer lifetime value and engagement metric: how much a user buys (volume) vs. how much they spend in dollars. Useful for loyalty segmentation or churn prediction.

---

15. Products Above Category Average Price (with full subtree)

What it does:
Within a category (including all its recursive subcategories), finds products priced above the average price of that category's products. Shows title, price, and stock.

Key techniques:

- Recursive CTE to collect category subtree like Query 1.
- Computation of the average price inside that category group (`avg_price`).
- Comparison `p.price > avg_price` filters premium/above-market items.
- Left join to inventory for stock.

Real-world purpose:
Identifying high-margin or luxury items in a category for highlighting ("premium picks") or to analyze pricing strategy relative to category norms.

---

16. Users Whose Cart Value Exceeds Average Order

What it does:
Finds users whose current cart total (sum of item quantity × price) is greater than the average order total across all orders. Returns username, cart value, and item count.

Key techniques:

- Computes `avg_order` separate from cart aggregation (`user_cart`).
- Cross join to compare each user's cart to the global average.
- Filtering ensures only "big" carts are surfaced.
- Rounds cart value.

Real-world purpose:
Cart prioritization: e.g., if a non-checked-out cart is more valuable than typical purchases, trigger email/SMS reminders or fraud review.

---

17. Delivered Orders to User's Default Address

What it does:
For each user with a default address, counts how many orders were delivered to that address and the total spent on them. Returns username, address id, order count, and money spent.

Key techniques:

- Filters for `is_default = true` to pick the preferred address.

- Joins to `orders` and only includes those with status `'Delivered'`.
- Aggregates per user/address pair and excludes zero-order cases via `HAVING`.

Real-world purpose:
Logistics/fulfillment reporting: verify delivery performance per default shipping location or validate that users are using their expected address. Can help detect address-related delivery issues.

---

18. Products in ≥2 Categories with Avg Rating ≥4

What it does:
Selects products that are assigned to at least two distinct categories and have an average rating of 4.0 or higher.

Key techniques:

- Subquery for category count with `HAVING COUNT(DISTINCT ...) >= 2`.
- Subquery for average rating with threshold.
- Join both to product to satisfy both conditions.

Real-world purpose:
Find versatile, well-rated products that cross category boundaries—ideal candidates for featured promotions ("popular across multiple sections") or bundles.

---

19. Trigger Logging Premium-Only Discount Usage

What it does:
When an order is inserted, if its discount is marked `premium_only = true`, writes a log entry to `premium_discount_usage`.

Key techniques:

- Adds a flag `premium_only` to the `discounts` table to distinguish special discounts.
- Trigger function checks both existence of `discount_id` and that the discount is premium-only before logging.
- Separate logging table keeps an audit trail. No incrementing counter here (unlike query 13).

Real-world purpose:
Track usage of exclusive promotions meant only for premium subscribers—for verifying eligibility enforcement and measuring the uptake of VIP offers.

---

20. Trigger Logging Product Price Changes

What it does:
Whenever a product's price changes, logs the old price, new price, and when the change happened.

Key techniques:

- `AFTER UPDATE OF price` with a `WHEN (OLD.price IS DISTINCT FROM NEW.price)` guard ensures only real changes are logged (no no-op updates).
- Logging table stores history; the function compares old vs new.
- `timestamptz` captures timezone-aware change timestamp.

Real-world purpose:
Price audit trail, rollback support, or observing dynamic pricing effects—essential for compliance, revenue analysis, and debugging pricing bugs.

---

21. Trigger Auto-assign First Address if Missing

What it does:
Before inserting an order, if no shipping address is provided (`NULL`), automatically fills in the user's default address or earliest address.

Key techniques:

- Trigger is `BEFORE INSERT` so it can mutate `NEW.shipping_address_id`.
- Ordering logic prefers `is_default = true` but falls back to oldest address deterministically.
- Raises an exception if the user has no address, preventing invalid orders.

Real-world purpose:
User experience improvement: reduce friction in order creation by auto-filling addresses. Also prevents data integrity issues by ensuring every order has a valid shipping address.

---

22. Products with Attribute in Carts across Users Above Average

What it does:
Finds products with a specific attribute (e.g., `Color='Black'`) that are in at least three different users' carts and whose total cart quantity exceeds the average cart quantity across all products. Returns title, attribute value, total cart quantity, stock, and distinct user count.

Key techniques:

- Aggregates per-product cart stats including distinct user count.
- Computes global average cart quantity and compares with each product.
- Filters for a minimum of three distinct users to focus on broader interest.
- Cross joins the average for easy comparison.

Real-world purpose:
Early signals of rising demand for specific variants—could inform preemptive inventory restocking or targeted marketing to users viewing those products.

---

23. Attribute Combination Products Ordered by Premium Users Above Their Average

What it does:
Identifies products with a given EAV attribute combination
(e.g., `Color='Black'` & `Weight='1.0'`) that have been ordered by at least two premium users, with total order value exceeding the average order value for premium users. Shows title, total value, number of premium users, and stock.

Key techniques:

- Filters premium-user orders separately and computes their average before comparing.
- Enforces both multi-user penetration (`COUNT(DISTINCT o.user_id) >= 2`) and value threshold.
- Joins inventory to show availability.

Real-world purpose:
Spot high-value, premium-user-approved SKUs that could be boosted (e.g., premium-exclusive bundles), or used to tailor offers to similar customers.

---

24. Products with an Attribute in Categories Having Above-Average Order Quantity

What it does:
Finds products with a specific attribute (e.g., `Color='Black'`) that belong to categories whose total order quantity is above the category average. Returns product title, category name, attribute value, total ordered quantity, and review count (only includes reviewed products).

Key techniques:

- Computes category-level total quantities and their average, filters "good" categories.
- Separately gathers per-product order totals and review counts.
- Joins the filtered attribute-specific products only if they sit in qualifying categories.
- Ensures at least one review via joining `review_counts`.

Real-world purpose:
Combines product attribute interest with category-level momentum and social proof to find "hot" items for promotion or deeper analysis.

Unique Approaches

- **Separation of concerns (e.g., query 14):** splitting aggregation (items vs. spend) into separate CTEs to avoid overcounting shows awareness of join-multiplication pitfalls.
- **Recursive CTEs (queries 1 & 15):** used to naturally traverse hierarchical data without manual loops.
- **Trigger guards:** many triggers use `WHEN` or conditional logic to avoid unnecessary work (e.g., only log on actual price change, only act when discount exists).
- **Audit logging + counter increment (query 13):** dual approach to both have quick metrics (`usage_count`) and a full detail log (`discount_usage_log`) is a robust pattern for observability.
- **Fallback logic (query 21):** deterministic selection of address with a clear priority order mitigates ambiguity in auto-assignment.